University Laboratory Report

# DVC Lab Report

Versioning a Sentiment Dataset with Git and DVC


| | |
|---|---|
| **Student Name:** | Tran Ly Nhat Hao |
| **Student ID:** | 23127187 |
| **Course:** | CSC15012 - Applications of NLP in Industry |
| **Instructor:** | Nguyen Hong Buu Long |


Project name: `sentiment-dvc`

# Contents


**1 Objective** **2**

**2 Project Structure** **2**

**3 Environment and Tools Used** **3**

**4 Main Commands Summary** **3**

**5 Step-by-Step Implementation** **3**

5.1 Project Creation . . . . . 3
5.2 Git Initialization . . . . . 4
5.3 DVC Initialization . . . . . 4
5.4 Adding the Dataset . . . . . 4
5.5 Connecting GitHub . . . . . 5
5.6 Connecting DVC Remote Storage . . . . . 6
5.7 Pushing Code and Data . . . . . 6

**6 Dataset Version 1** **7**

**7 Dataset Version 2** **7**

**8 Retrieve Dataset on Another Machine** **8**

8.1 Standard Workflow . . . . . 8
8.2 Google Colab Example . . . . . 8

**9 List and Switch Between Dataset Versions** **9**

9.1 Relation Between Git and DVC . . . . . 9
9.2 List Available Versions . . . . . 9
9.3 Switch to a Specific Version . . . . . 10

**10 Corrections to Mistakes or Unclear Points in the Original Lab Instructions** **10**

**11 Conclusion** **11**

## 1. Objective

The objective of this laboratory exercise is to use Data Version Control (DVC) to manage versions of a small sentiment analysis dataset in a reproducible project named `sentiment-dvc`. The practical work consists of creating the project structure, initializing Git and DVC, connecting a Git repository and a DVC remote storage service, tracking the first dataset version, updating the dataset to create version 2, and documenting how dataset versions can be retrieved and switched on another machine.

## 2. Project Structure

The repository used in this lab contains the following essential files and directories:

```
sentiment-dvc/
|-- .dvc/
|-- data/
|   `-- raw/
|       |-- .gitignore
|       |-- sentiment.csv
|       `-- sentiment.csv.dvc
|-- screenshots/
|   `-- .gitkeep
|-- .dvcignore
|-- .gitignore
|-- README.md
`-- report.tex
```

The main dataset file is stored at `data/raw/sentiment.csv`, while DVC tracks it through the pointer file `data/raw/sentiment.csv.dvc`.

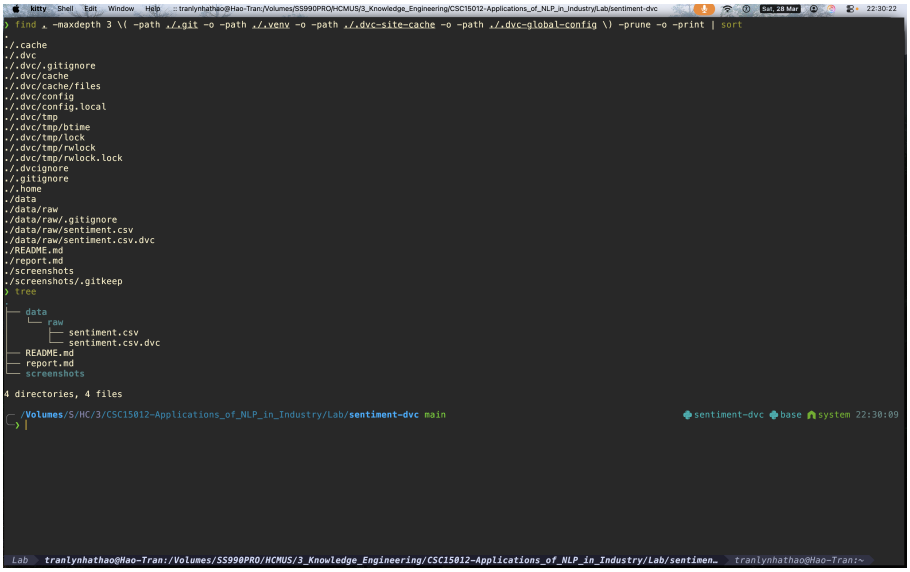


Figure 1: Project structure

## 3. Environment and Tools Used

The local environment used for this lab was the terminal workspace in which the project was created and verified. The main tools recorded in the repository are listed below.

- Git version: `2.53.0`
- Python version: `3.12.13`
- DVC version: `3.67.0`
- GitHub repository: https://github.com/tranlynhathao/sentiment-dvc.git
- DagsHub Git repository: https://dagshub.com/tranlynhathao/sentiment-dvc.git
- DVC remote endpoint: https://dagshub.com/tranlynhathao/sentiment-dvc.s3

At the time this report was prepared, the local `main` branch pointed to commit `f03720e`, and both `origin/main` and `dagshub-git/main` referenced the same commit.

## 4. Main Commands Summary

Table 1 summarizes the commands that were central to the lab workflow.

| Command | Purpose |
|---|---|
| `git init -b main` | Initialize the Git repository on the `main` branch. |
| `dvc init` | Initialize DVC inside the project. |
| `dvc add data/raw/sentiment.csv` | Track the dataset with DVC. |
| `git add ...` and `git commit` | Record project and dataset changes in Git. |
| `git push` | Publish code history to a Git remote. |
| `dvc push` | Upload DVC-tracked data objects to the remote storage. |
| `git log -- data/raw/sentiment.csv.dvc` | Inspect the history of dataset versions through the DVC pointer file. |
| `git checkout <revision>` | Move to a specific Git revision. |
| `dvc checkout` or `dvc pull` | Materialize the dataset version referenced by the active Git revision. |

Table 1: Main commands used in the DVC lab.

## 5. Step-by-Step Implementation

### 5.1. Project Creation

The project was created with a directory structure centered around a small CSV file for sentiment classification. The initial dataset contents were as follows:

```
text,label
"I love this movie",positive
"This was terrible",negative
"Pretty good overall",positive
"Waste of time",negative
```

This file was saved as `data/raw/sentiment.csv`.

### 5.2. Git Initialization

Git was initialized in the project root with the `main` branch.

```
git init -b main
git add .dvc .dvcignore .gitignore screenshots/.gitkeep
git commit -m "Initialize Git and DVC"
```

The initial repository setup is represented by commit `feaec47`.

### 5.3. DVC Initialization

DVC was initialized after Git so that repository metadata such as `.dvc/config`, `.dvc/.gitignore`, and `.dvcignore` could be created.

```
dvc init
```

These files establish the DVC configuration and ensure that local cache and local-only configuration files are not committed into Git accidentally.

Figure 2: Git and DVC initialization

### 5.4. Adding the Dataset

The dataset was added to DVC using the following command:

```
dvc add data/raw/sentiment.csv
```

This command created the file `data/raw/sentiment.csv.dvc` and the ignore rule `data/raw/.gitignor`
Git then tracked those generated files instead of tracking the CSV directly.

```
git add data/raw/.gitignore data/raw/sentiment.csv.dvc
git commit -m "Track sentiment dataset v1"
```

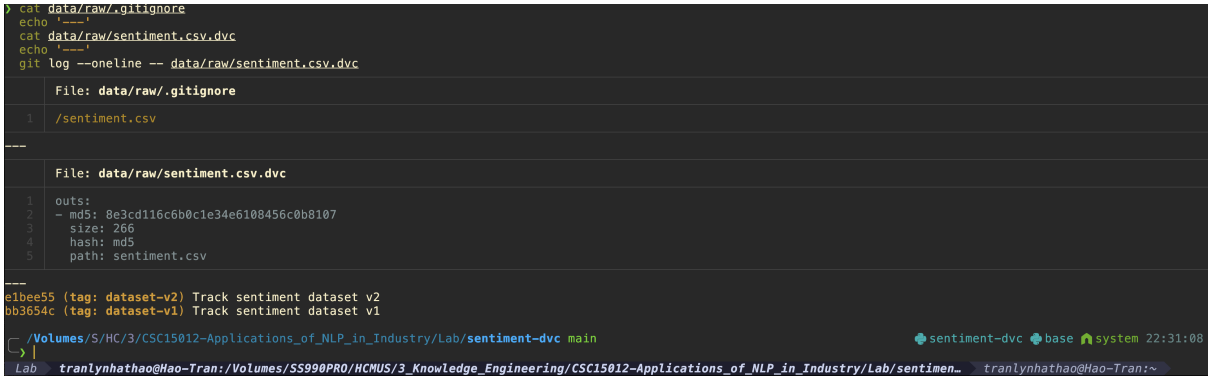


Figure 3: dvc add result

## 5.5. Connecting GitHub

The code repository was connected to GitHub through the remote named `origin`. In the final repository state, the configured URL was:

```
https://github.com/tranlynhathao/sentiment-dvc.git
```

A typical sequence for connecting GitHub is shown below.

```
git remote add origin https://github.com/tranlynhathao/sentiment-dvc.git
git push -u origin main
git push origin --tags
```

The first push uses `-u` to associate the local `main` branch with `origin/main`. Subsequent pushes can then be performed with a simple `git push`.

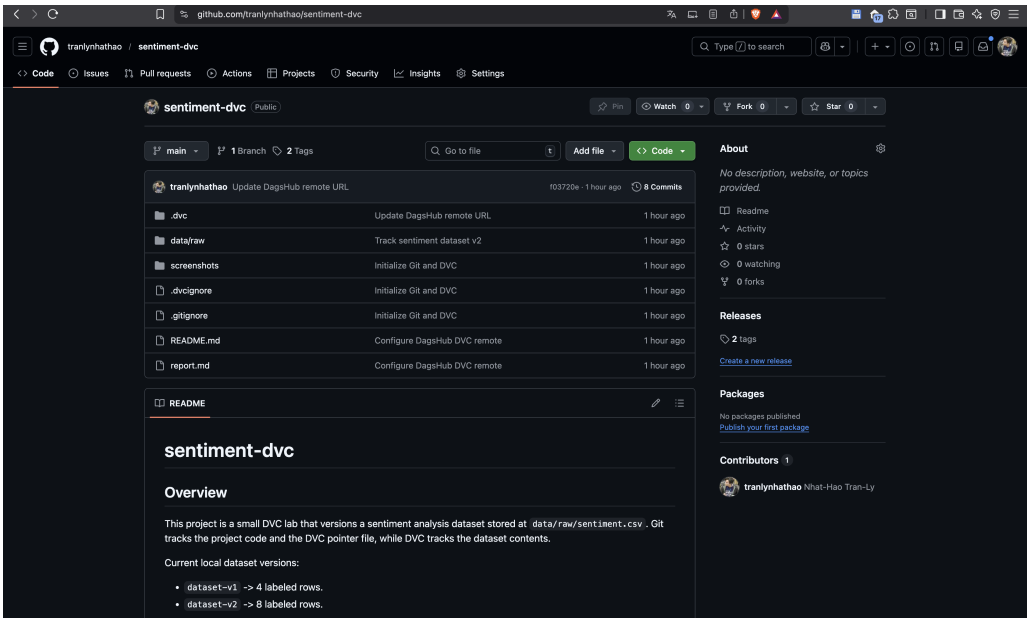


Figure 4: GitHub repository

### 5.6. Connecting DVC Remote Storage

The DVC remote was configured to use DagsHub through its S3-compatible endpoint. The repository currently stores the following DVC remote settings in `.dvc/config`:

```
[core]
    remote = dagshub
['remote "dagshub"']
    url = s3://dvc
    endpointurl = https://dagshub.com/tranlynhathao/sentiment-dvc.s3
```

The generic commands used for this setup are:

```
dvc remote add -d dagshub s3://dvc
dvc remote modify dagshub endpointurl https://dagshub.com/tranlynhathao/
   sentiment-dvc.s3
```

Credentials must be stored locally rather than in tracked Git files. A safe workflow is:

```
dvc remote modify --local dagshub access_key_id <DAGSHUB_TOKEN>
dvc remote modify --local dagshub secret_access_key <DAGSHUB_TOKEN>
```

These commands write to `.dvc/config.local`, which is ignored by Git.

Figure 5: DVC remote configuration

### 5.7. Pushing Code and Data

Once the Git and DVC remotes are configured, code and data can be uploaded with the following commands:

```
git push origin main
git push origin --tags
git push -u dagshub-git main
git push dagshub-git --tags
dvc push -r dagshub
```

In this lab, Git remotes were configured for both GitHub and DagsHub. DVC data upload required local authentication through a DagsHub access token. The standard `dvc push` command uploads data for the active revision. If every historical dataset version referenced by all commits must be uploaded, the safer command is:

```
dvc push -r dagshub --all-commits
```

This point is important because the original assignment text implies that both dataset versions should remain available, but it does not explicitly mention `-all-commits`.

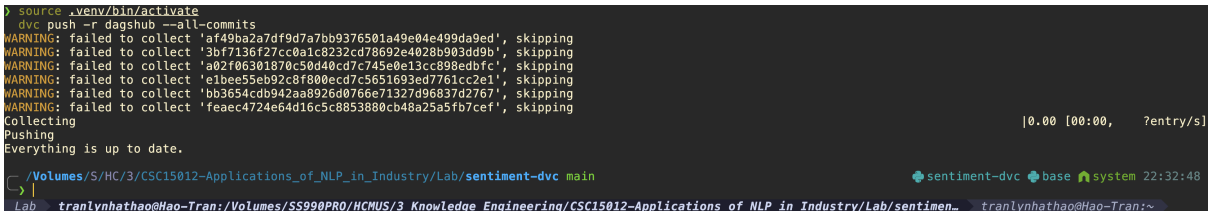


Figure 6: dvc push result

## 6. Dataset Version 1

Dataset version 1 corresponds to the state committed in `bb3654c` and tagged as `dataset-v1`. The CSV file contained four labeled examples. Its DVC pointer file recorded the following metadata:

```
outs:
- md5: 7bda551b49fe980c83ae5d85b146f1cc
  size: 125
  hash: md5
  path: sentiment.csv
```

This version establishes the baseline dataset used in the remainder of the exercise.

## 7. Dataset Version 2

Dataset version 2 was created by extending the dataset with four additional rows while preserving the same schema. The additional examples were:

```
"Absolutely fantastic acting",positive
"The plot was engaging",positive
"I would not recommend this",negative
"Too long and boring",negative
```

The updated version was committed in `e1bee55` and tagged as `dataset-v2`. Its DVC pointer metadata is shown below:

```
outs:
- md5: 8e3cd116c6b0c1e34e6108456c0b8107
  size: 266
  hash: md5
  path: sentiment.csv
```

The current workspace corresponds to this second dataset version.

Figure 7: Dataset version 2 update

# 8. Retrieve Dataset on Another Machine

## 8.1. Standard Workflow

To retrieve the dataset on another machine, the repository must first be cloned, then DVC must be installed, and finally the data must be pulled from the configured remote. The general workflow is:

```
git clone https://github.com/tranlynhathao/sentiment-dvc.git
cd sentiment-dvc
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install "dvc[s3]"
dvc remote modify --local dagshub access_key_id <DAGSHUB_TOKEN>
dvc remote modify --local dagshub secret_access_key <DAGSHUB_TOKEN>
git checkout main
dvc pull -r dagshub
```

The key idea is that Git retrieves repository history and the DVC pointer files, while DVC retrieves the actual dataset contents.

## 8.2. Google Colab Example

A similar workflow can be used on Google Colab. Commands can be executed in a shell cell or prefixed with `!`.

```
git clone https://github.com/tranlynhathao/sentiment-dvc.git
cd sentiment-dvc
pip install "dvc[s3]"
dvc remote modify --local dagshub access_key_id <DAGSHUB_TOKEN>
dvc remote modify --local dagshub secret_access_key <DAGSHUB_TOKEN>
git checkout main
dvc pull -r dagshub
python - <<'PY'
import pandas as pd
print(pd.read_csv('data/raw/sentiment.csv'))
```

```
PY
```

The usual rule for another machine is:

1. obtain the correct Git revision first, and then
2. run `dvc pull` to materialize the data referenced by that revision.

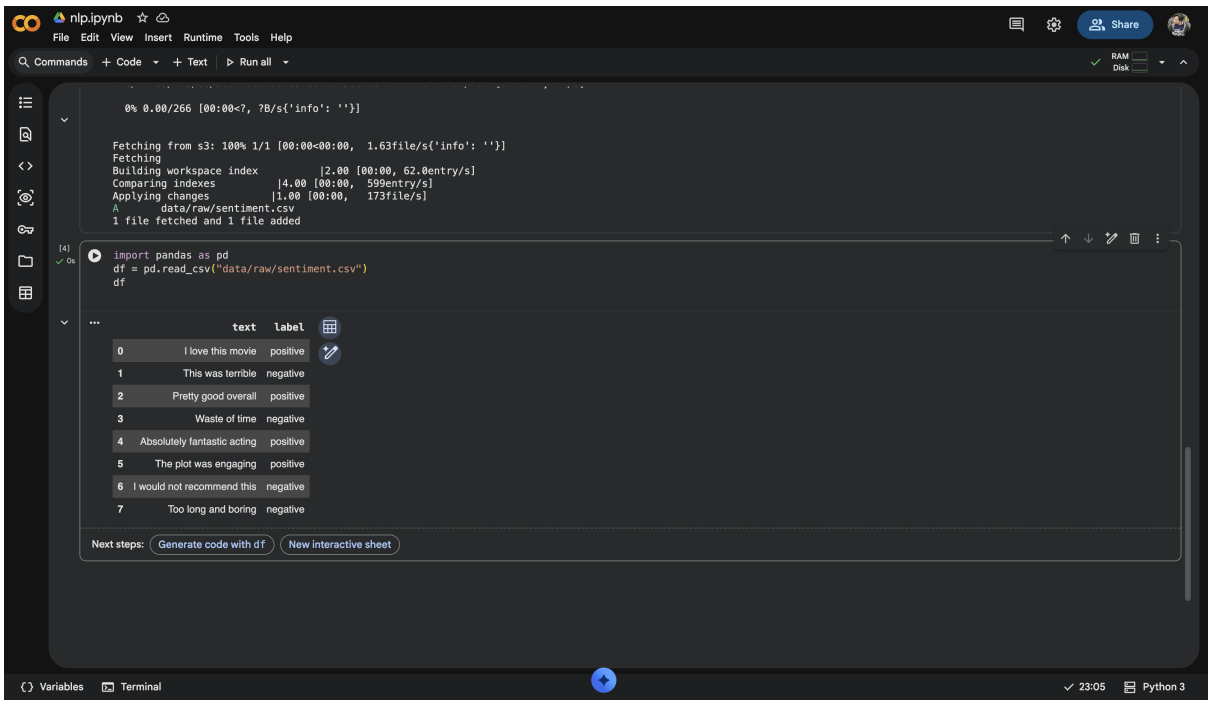


Figure 8: Retrieving dataset on another machine

# 9. List and Switch Between Dataset Versions

## 9.1. Relation Between Git and DVC

Git and DVC play different roles in version management. Git stores the version history of the lightweight pointer file `data/raw/sentiment.csv.dvc`, while DVC stores the actual dataset content in cache and remote storage. For this reason, switching dataset versions is not done by DVC alone. Git must first move the repository to the commit or tag associated with the desired version.

## 9.2. List Available Versions

Dataset history can be inspected through the DVC pointer file and through Git tags.

```
git tag --list
git log --oneline -- data/raw/sentiment.csv.dvc
```

Expected output includes the two dataset tags and the two dataset commits:

```
dataset-v1
dataset-v2

e1bee55 Track sentiment dataset v2
bb3654c Track sentiment dataset v1
```

### 9.3. Switch to a Specific Version

If version 1 is required and the data is already present in the local DVC cache, the following commands are sufficient:

```
git checkout dataset-v1
dvc checkout
```

If the desired version is not available in the local cache, the second command should instead download it from the remote:

```
git checkout dataset-v1
dvc pull -r dagshub
```

To return to the latest version on the `main` branch:

```
git checkout main
dvc checkout
```

In practical use, the rule is simple: first select the Git revision, then restore the data with DVC.

## 10. Corrections to Mistakes or Unclear Points in the Original Lab Instructions

Several parts of the original lab wording are incomplete or potentially misleading.

1. The DagsHub remote URL shown in the original instructions is outdated. Current DagsHub usage relies on an S3-compatible endpoint, not the older `https://dagshub.com/.../.dvc` form.
2. DagsHub credentials should not be written into tracked Git configuration. They should be added with `dvc remote modify -local ...`, which stores them in `.dvc/config.local`.
3. A first Git push normally needs `git push -u origin main` rather than a plain `git push`.
4. Cloning the repository on another machine is not enough to recover the dataset. After selecting the correct Git revision, `dvc pull` is required.
5. Switching versions is a two-step process. `git checkout` changes the revision, while `dvc checkout` or `dvc pull` restores the actual dataset file.

Figure 9: Switching between dataset versions

6. If all historical dataset versions must be available on the DVC remote, `dvc push -all-commits` should be considered. The original instructions do not state this clearly.

## 11. Conclusion

This lab demonstrates a practical workflow for dataset version control using Git and DVC. The `sentiment-dvc` project stores the dataset in `data/raw/sentiment.csv`, tracks versions through `data/raw/sentiment.csv.dvc`, and uses Git commits and tags to identify meaningful dataset states. The exercise also shows that dataset retrieval on another machine depends on both tools: Git selects the correct revision, and DVC restores the matching data.

From a reproducibility perspective, the most important lesson is that DVC should not be treated as a replacement for Git. Instead, the two systems work together: Git manages the project history, while DVC manages large data artifacts and their remote storage. This division of responsibility makes it possible to reproduce both version 1 and version 2 of the dataset in a controlled and auditable manner.